

# Manual do Usuário — Simulador de Gerenciamento de Memória (TP2)

---

Sistemas Operacionais — Trabalho Prático 2.

Simulador de alocação contígua de memória com dois esquemas:

- **(i) Particionamento variável** — políticas **Worst-Fit** ou **Circular-Fit**.
- **(ii) Sistema Buddy** — divisão/coalescência em potências de 2, com relatório de fragmentação interna.

A cada requisição lida do arquivo, a ferramenta mostra o estado da memória, os blocos contíguos livres e (no Buddy) a fragmentação interna total.

---

## 1. Requisitos

---

- **Python 3.7 ou superior** (não há dependências externas — apenas biblioteca padrão).

Verifique a versão:

```
python --version
```

Nos laboratórios, caso `python` aponte para Python 2, use `python3`.

---

## 2. Estrutura dos arquivos

---

```
T2-S0/
├── src/
│   ├── main.py           # ponto de entrada (menu, parser, visualização)
│   ├── memoria_variavel.py # Worst-Fit e Circular-Fit
│   └── memoria_buddy.py  # Sistema Buddy
├── entradas/
│   ├── exemplo_variavel.txt # exemplo do enunciado (memória 16, worst-fit)
│   ├── exemplo_buddy.txt   # exemplo do Buddy (memória 256)
│   └── exemplo_tabela1.txt  # sequência da Tabela 1 do enunciado
├── MANUAL.md              # este manual
└── doc/TP2_2026.pdf       # enunciado
```

Como o programa é em Python, **não há etapa de compilação**. Basta executar.

---

## 3. Formato do arquivo de requisições

---

Arquivo texto, uma requisição por linha:

| Comando                                    | Significado                                                     |
|--------------------------------------------|-----------------------------------------------------------------|
| <code>IN(ID, tam)</code>                   | aloca <code>tam</code> posições para o processo <code>ID</code> |
| <code>OUT(ID)</code>                       | libera o espaço ocupado pelo processo <code>ID</code>           |
| <code># ...</code>                         | comentário (linha ignorada)                                     |
| <code>256</code> ou <code>\  256 \ </code> | tamanho da memória (opcional, dentro do arquivo)                |

Os espaços são livres: `IN(A, 10)` e `IN(A,10)` são aceitos. Maiúsc./minúsc. de `IN/OUT` não importam. Também é aceito o formato **sem parênteses**: `IN A 10`, `IN A, 10`, `OUT A`.

Os identificadores de processo (`ID`) diferenciam maiúsculas de minúsculas: `A` e `a` são processos distintos. Mantenha a mesma grafia no `IN` e no `OUT`.

O tamanho da memória deve ser sempre uma potência de 2 (4, 8, 16, 32, 64, 128, 256, ...).

Exemplo (`entradas/exemplo_variavel.txt`):

```
16
IN(A, 3)
IN(B, 2)
IN(C, 1)
OUT(A)
IN(D, 3)
OUT(B)
OUT(C)
OUT(D)
```

## 4. Como executar

A partir da pasta `src/`:

```
cd src
```

### 4.1 Modo interativo (recomendado)

```
python main.py
```

O programa pergunta:

1. Esquema — `1` partições variáveis ou `2` sistema buddy;
2. (Se variável) política — `1` Worst-Fit ou `2` Circular-Fit;
3. Caminho do arquivo de requisições;
4. Tamanho da memória (potência de 2 — se o arquivo já trouxe um número, ele é sugerido);
5. Modo passo a passo (`s` pausa a cada linha; Enter avança).

### 4.2 Modo por linha de comando (lote)

```
python main.py --tipo variavel --politica worst --memoria 16 --arquivo ../entradas/exemplo_variavel.txt
python main.py --tipo variavel --politica circular --memoria 16 --arquivo ../entradas/exemplo_variavel.txt
python main.py --tipo buddy --memoria 256 --arquivo ../entradas/exemplo_buddy.txt
python main.py --tipo buddy --memoria 64 --bloco-min 4 --arquivo ../entradas/exemplo_buddy.txt
```

Opções:

| Opção                    | Valores                                    | Descrição                                          |
|--------------------------|--------------------------------------------|----------------------------------------------------|
| <code>--tipo</code>      | <code>variavel</code>   <code>buddy</code> | esquema de alocação                                |
| <code>--politica</code>  | <code>worst</code>   <code>circular</code> | política (só para <code>variavel</code> )          |
| <code>--memoria</code>   | inteiro potência de 2                      | tamanho total da memória                           |
| <code>--bloco-min</code> | potência de 2 (padrão 1)                   | menor bloco do buddy (só para <code>buddy</code> ) |
| <code>--arquivo</code>   | caminho                                    | arquivo de requisições                             |
| <code>--passo</code>     | (sem valor)                                | pausa a cada passo, aguardando Enter               |

Se `--memoria` for omitido, usa-se o número informado no arquivo.

## 5. Lendo a saída

Para cada passo são exibidos:

```
[5] IN(D, 3)
Mapa : [.....#####.....]
Segm. : (3)[B:2][C:1][D:3](7)
Livres: | 3 | 7 | (2 bloco(s) contiguo(s) livre(s), total 10)
```

- **Mapa** — barra proporcional da memória: `#` = ocupado, `.` = livre.
- **Segm.** — segmentos em ordem de endereço: `[ID:tam]` ocupado, `(tam)` livre.
- **Livres** — lista dos blocos **contíguos livres** (formato `| t1 | t2 | ... |` do enunciado) e a contagem total.

No **Sistema Buddy** acrescenta-se:

```
Buddy livres: | 32 | 64 | 128 |
Fragmentacao interna total: 11
```

- **Buddy livres** — blocos livres do alocador buddy (cada um potência de 2).
- **Fragmentacao interna total** — soma de `(tamanho do bloco - tamanho pedido)` de todos os processos alocados.

Quando uma alocação não cabe na memória, o programa imprime:

```
>>> ESPACO INSUFICIENTE DE MEMORIA (processo 'C' nao alocado)
```

e segue para a próxima requisição.

---

## 6. Detalhes das políticas e do Buddy

---

- **Worst-Fit** — aloca no **maior** bloco livre disponível; em caso de empate, no de menor endereço.
- **Circular-Fit (Next-Fit)** — procura o primeiro bloco livre que caiba a partir da posição da última alocação, percorrendo a memória de forma circular.
- **Coalescência (variável)** — ao liberar um processo, blocos livres fisicamente adjacentes são fundidos.
- **Buddy** — o pedido é arredondado para a próxima potência de 2 (ex.: 21 → 32). Blocos são divididos pela metade até o tamanho desejado; ao liberar, um bloco se funde com seu companheiro ( $\text{buddy} = \text{offset} \text{ XOR } \text{tamanho}$ ) enquanto ambos estiverem livres.
- **Bloco mínimo do buddy** (`--bloco-min`, padrão 1) — define o menor bloco alocável. Com `--bloco-min 4`, uma requisição de 1 ocupa um bloco de 4 (fragmentação interna 3) e nenhum bloco menor que 4 é criado.

---

## 7. Exemplo de validação (enunciado)

---

Memória 16, Worst-Fit, arquivo `exemplo_variavel.txt`. A coluna de blocos livres reproduz o enunciado:

```
13 -> 11 -> 10 -> 3 10 -> 3 7 -> 5 7 -> 6 7 -> 16
```

---

## 8. Robustez e testes

---

O programa tolera variações de formato no arquivo de requisições:

- Maiúsc./minúsc.: `IN`, `in`, `Out`, `OUT`.
- Espaços livres: `IN( A , 10 )` ou `IN(A,10)`.
- IDs de qualquer tamanho (`A`, `P1`, `proc_longo`).
- Linhas em branco e comentários (`# ...`) ignorados.
- Tamanho da memória como `256` ou `| 256 |`.
- Arquivos salvos pelo Bloco de Notas (BOM UTF-8) e quebras de linha Windows/Unix.

Situações de erro são tratadas **sem interromper** a simulação:

| Situação                               | Comportamento                                       |
|----------------------------------------|-----------------------------------------------------|
| Alocação não cabe (sem bloco contíguo) | imprime <code>ESPACO INSUFICIENTE DE MEMORIA</code> |

|                                              |                                           |
|----------------------------------------------|-------------------------------------------|
| Requisição maior que a memória               | ESPAÇO INSUFICIENTE DE MEMÓRIA            |
| Fragmentação externa (livre disperso)        | ESPAÇO INSUFICIENTE DE MEMÓRIA            |
| Fragmentação do buddy (sem bloco do tamanho) | ESPAÇO INSUFICIENTE DE MEMÓRIA            |
| OUT de processo inexistente / já liberado    | aviso e segue para a próxima linha        |
| IN de ID já ativo                            | aviso (um processo não existe duas vezes) |
| Linha não reconhecida                        | aviso e segue                             |

Arquivos de demonstração:

- `entradas/teste_frag_externa.txt` — fragmentação externa (variável).
- `entradas/teste_buddy_frag.txt` — fragmentação do buddy.
- `entradas/teste_formato_estranho.txt` — formato baguncado (parser).

## Suite automatizada

```
python tests/test_simulador.py
```

Executa ~48 verificações (enunciado worst-fit, circular-fit com wrap, buddy da Figura 1, coalescência, fragmentação interna/externa, reuso de ID, double free, potências de 2). Sai com código 0 se tudo passar.